

Identitäten statt Netzwerkgrenzen: SPIFFE/SPIRE in Zero-Trust-Architekturen



Stephan Kraft
Open Source Community
Advocate & Business
Development, Red Hat



Höchst, 17.März 2026

Common Approaches for Zero Trust Architecture¹

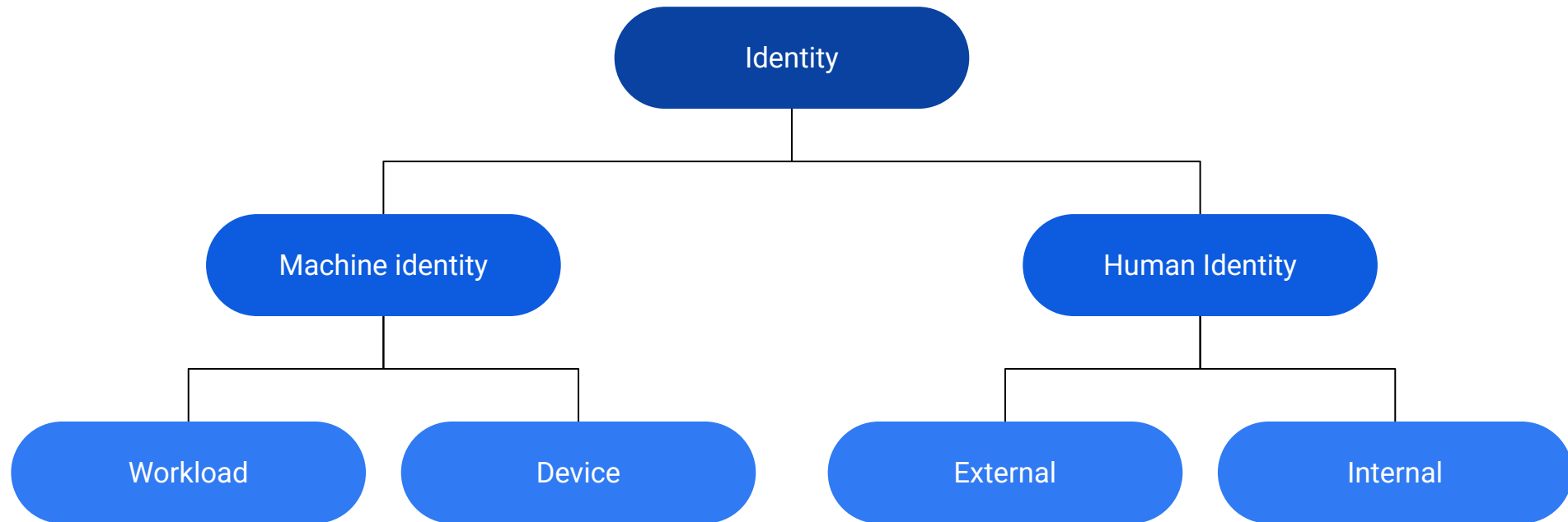
- (Perimeter-Security)
- Micro-Segmentation
- Software Defined Perimeter
- Enhanced Identity Governance

¹ A perimeter-based security model with a traditional firewall is not seen as a valid approach for a Zero Trust Architecture

Agenda

- ▶ Why Workload Identity?
- ▶ What is the common Anti-Pattern? And how can we improve?
- ▶ Introducing SPIFFE/SPIRE (Demo)
- ▶ Example SPIRE Use-Cases

What types of identity do exist?



Why are we discussing this topic?

- ▶ 23,7 Mio secrets detected in public GitHub commits ¹
- ▶ 72% of organizations had at least one certificate-related outage in the past year ²
- ▶ 50% of organizations reported security incidents that are related to compromised machine identities in the past year ²
- ▶ The average expectations for identities to grow in the next 12 month is 240% ³

Sources:

¹ "State of Secrets Sprawl Report 2025", GitGuardian: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2025>

² State of Machine Identity Security Report", CyberArk: <https://www.cyberark.com/CyberArk-2025-state-of-machine-identity-security-report.pdf>

³ "Identity Security Threat Landscape Report 2024, CyberArk: <https://www.cyberark.com/resources/ebooks/identity-security-threat-landscape-2024-report>



Software Supply Chain Attacks and pending legislation

- ▶ EU [Network and Information Security 2 Directive](#)
- ▶ EU [CRA \(Cyber Resilience Act\)](#)
- ▶ Government willingness to enforce and fine executives ignoring SSC
 - [SEC fines SolarWinds](#) and CISO for concealing vulnerabilities
 - [Log4J vulnerability](#)
- ▶ US Executive Order on Improving the Nation's [Cybersecurity](#)
- ▶ US Executive Order 14017 - [America's Supply Chains](#)
- ▶ US Executive order 14018 [Improving the Nation's Cybersecurity](#)



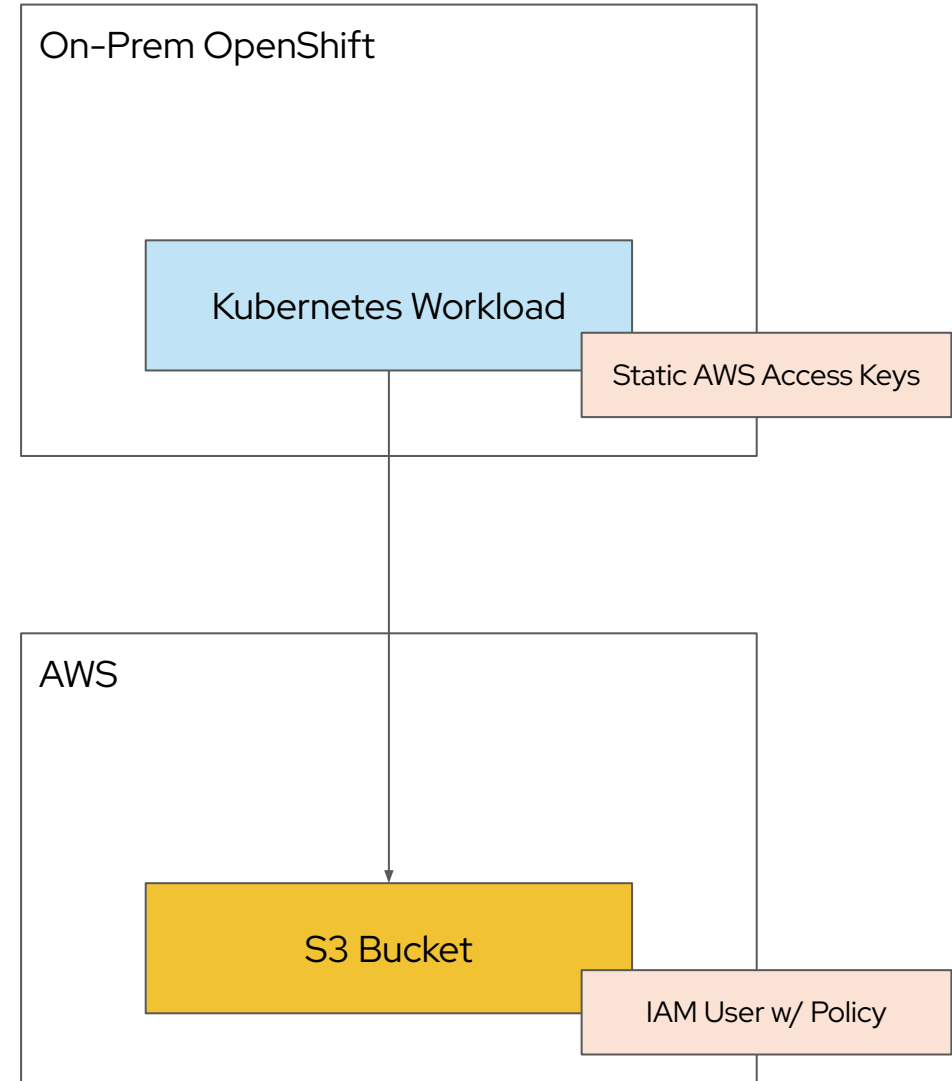
The Anti-Pattern



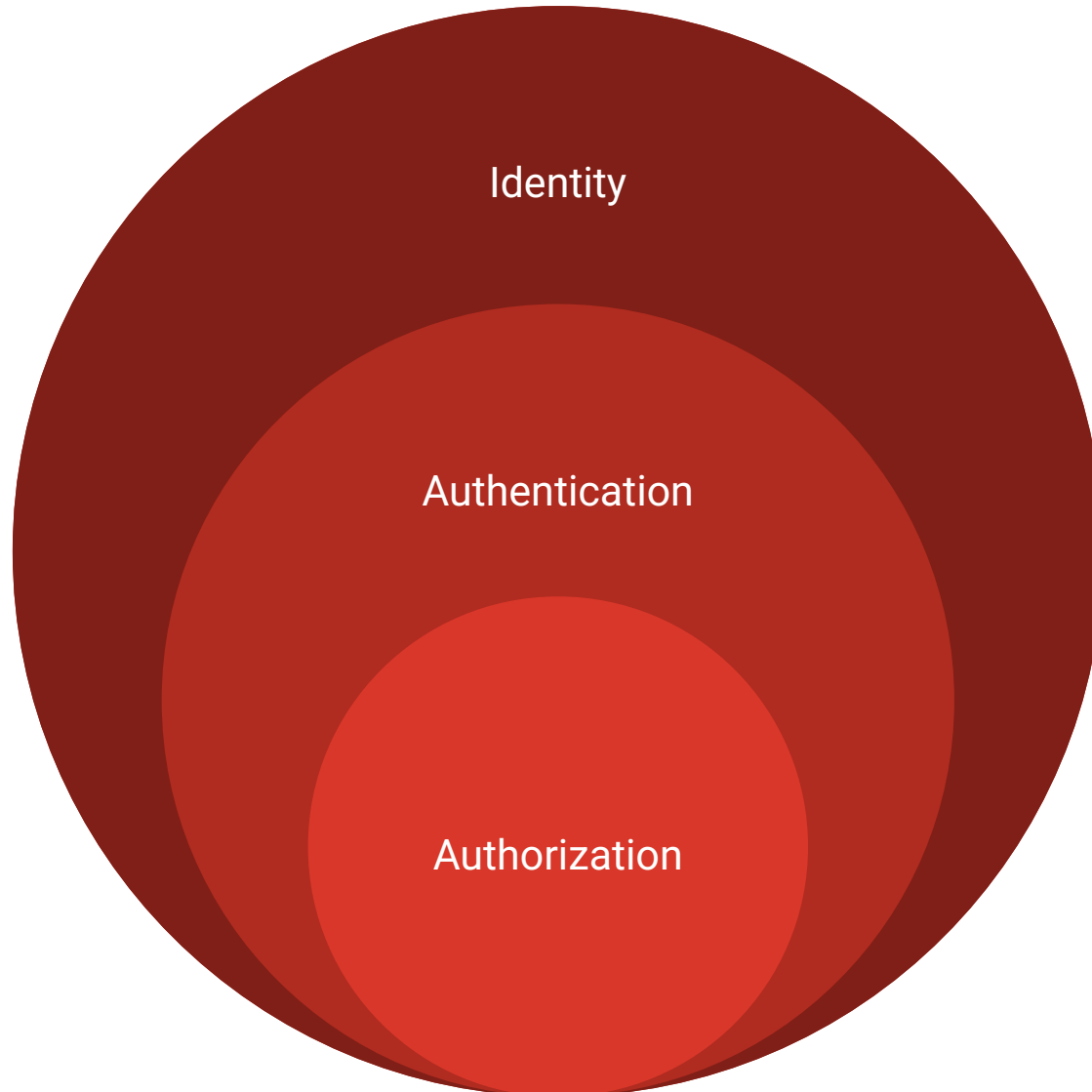
Anti-Pattern: Static Credentials

Example:

- ▶ On-prem OpenShift cluster hosts the Kubernetes workload.
- ▶ Workload uses static AWS keys for S3 access.
- ▶ An IAM User with a policy enables the access.



Anti-Pattern: Static credentials



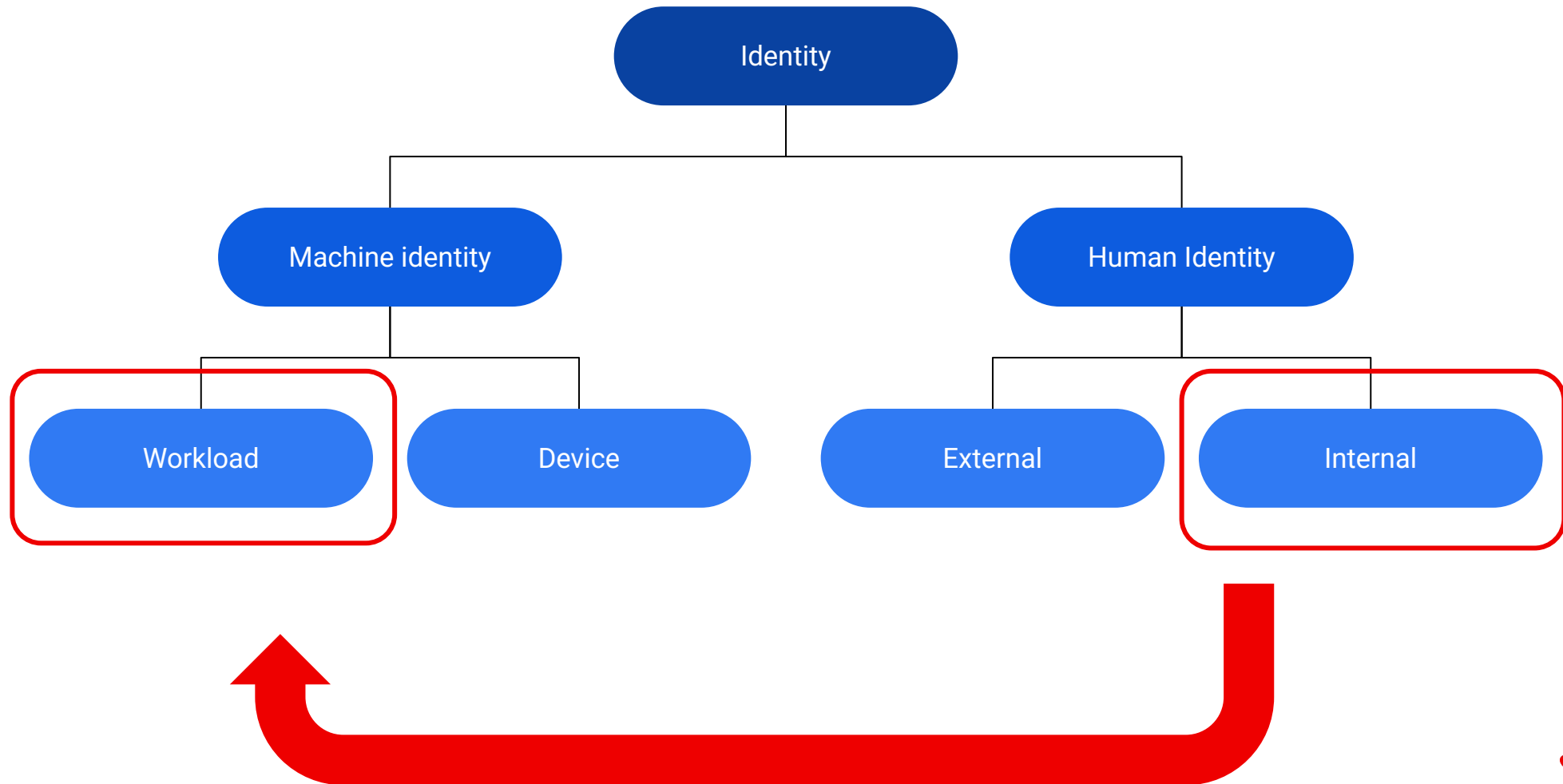
AWS IAM User

Static IAM Access Keys

AWS IAM Policy



Anti-Pattern: We use Human Identities for a Workload Identity scenario

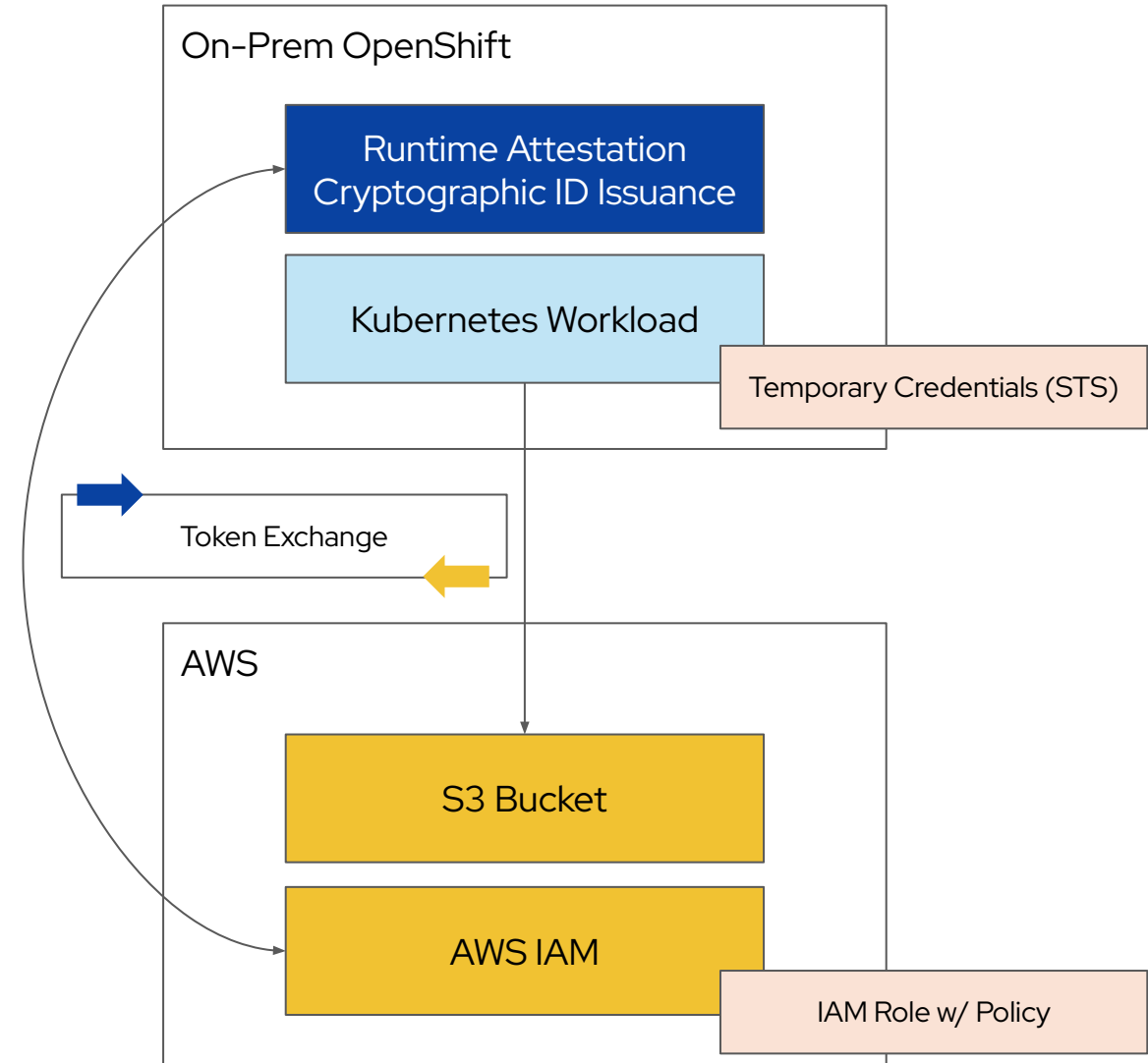


How Can We Improve?

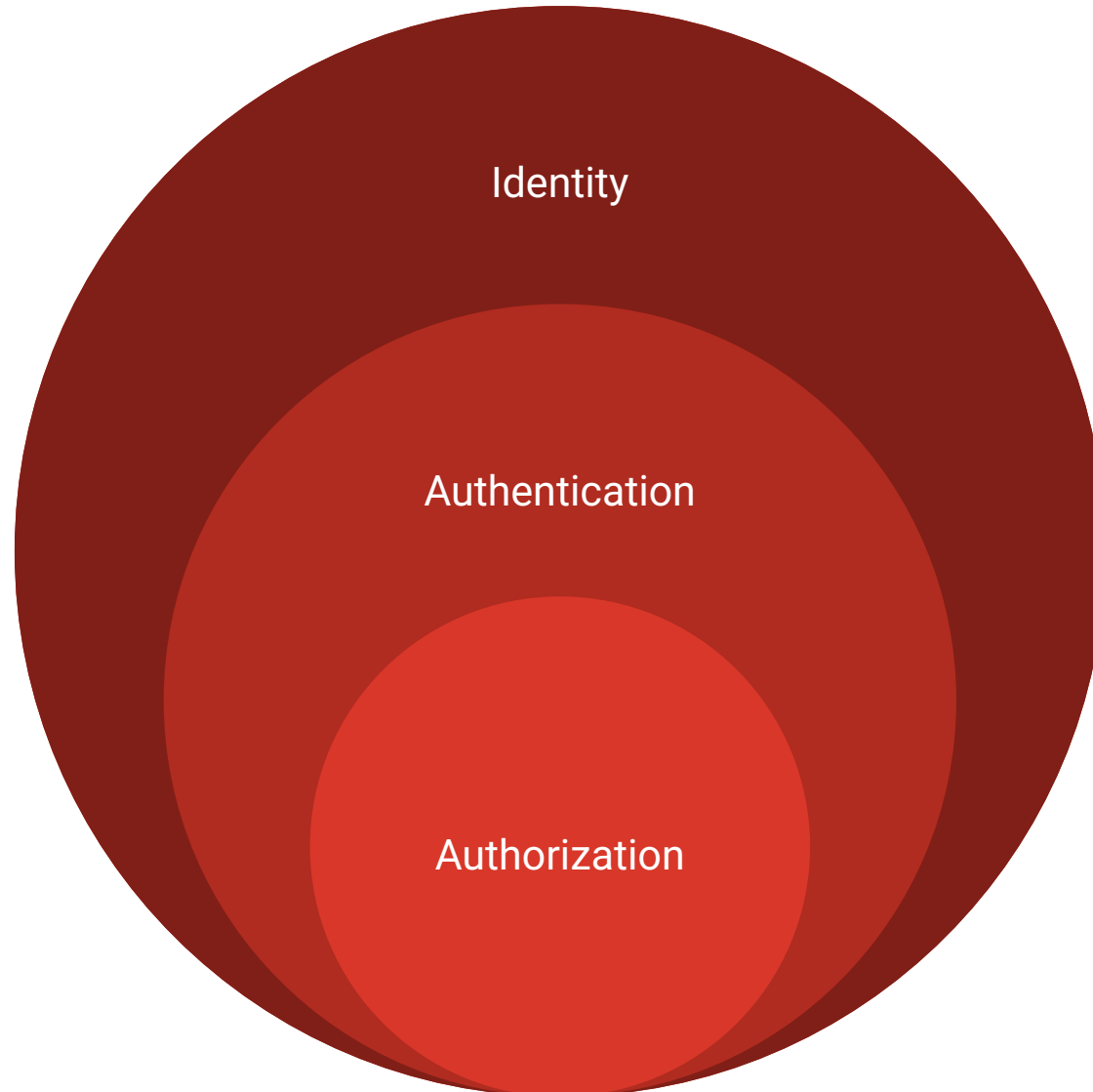


Best practice: Using Workload Identities with short-lived credentials

- ▶ Agent attests workload and requests a Cryptographic ID.
- ▶ Server issues a short-lived Cryptographic ID for workload identity (JWT/Certificate).
- ▶ Workload uses Cryptographic ID to assume an AWS IAM Role via OIDC Federation.
- ▶ Assumed IAM Role grants temporary access to the S3 Bucket.



What components were utilized?



Verifiable Workload
Attributes

Short-Lived Cryptographic
Token (JWT)

Federated IAM Role with
Policy



What Have We Accomplished?

- ▶ Substitute short-lived, automatically rotated credentials for hard-coded, long-lived secrets.
- ▶ Provide a **standardized identity control plane** for hybrid cloud environments.
- ▶ Establish **identity at the application level**, moving beyond reliance on for example IP:Port.



Introducing SPIFFE/SPIRE



SPIFFE and SPIRE

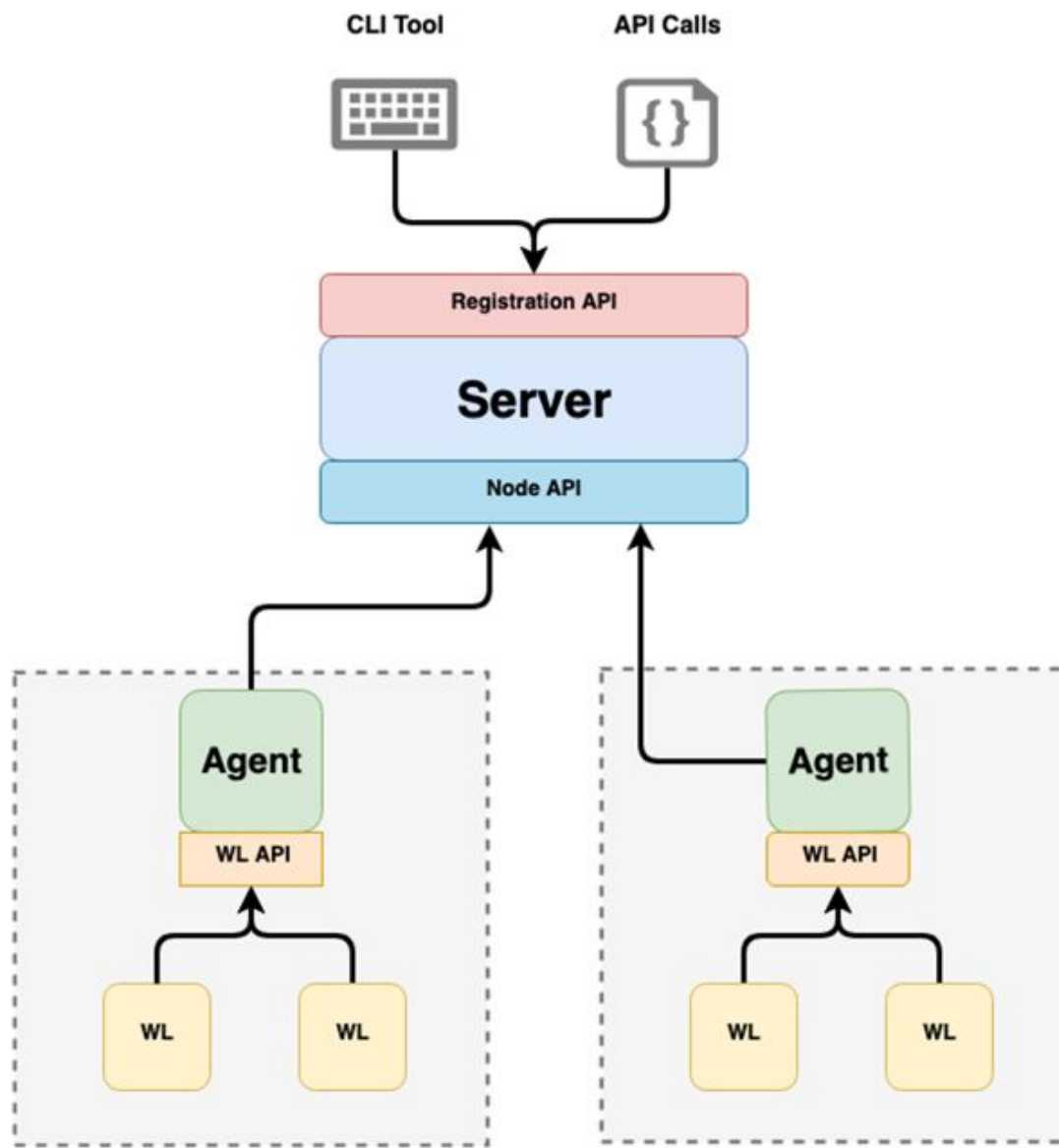


- ▶ SPIFFE is the standard for workload identity, implemented by SPIRE.
- ▶ Concept proposed by Joe Beda in 2016; originally developed by Scytale (now HPE) around 2017.
- ▶ CNCF project since 2018, Graduated in 2022.
- ▶ Strong community and adoption (e.g., Bloomberg, Bytedance, Uber, Netflix, Github).
- ▶ Integrated with products like cosign/sigstore, envoy, istio, OPA, and Hashicorp Vault.
- ▶ Pluggable architecture supports diverse environments (Linux/Unix, Windows, major clouds) and languages (Go, Java, Python, Rust, C++, .net).





SPIFFE/SPIRE



"We wanted to shift away from secret management practices in general. Instead of asking a service, 'what do you have,' we want to ask, 'who are you'".

(Healthcare Organization, who has implemented Managed Identities)

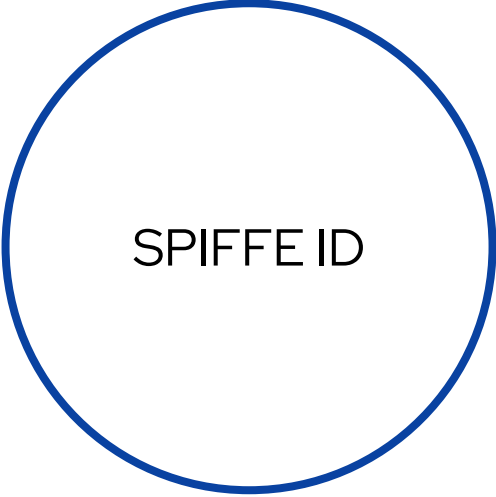
Source: <https://www.gitguardian.com/files/managed-identities-book>

SPIFFE/SPIRE is secretless,
Multi-factor-authentication
for workloads

Executive

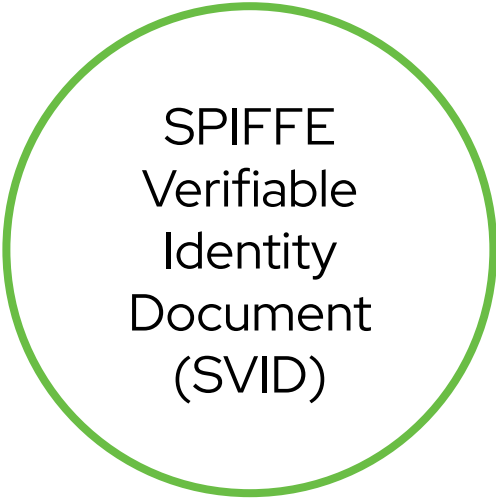
Technical Expert

Three Core Components of SPIRE



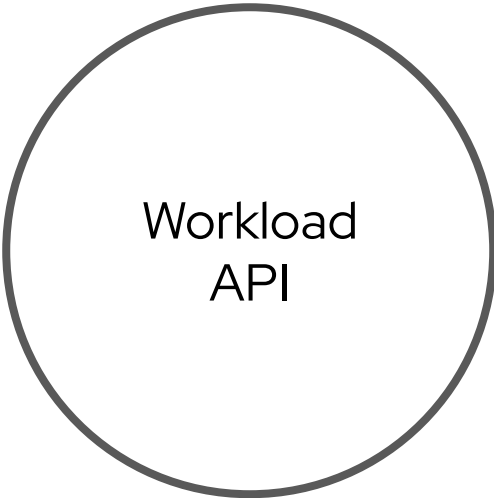
SPIFFE ID

String to uniquely identify
a workload



SPIFFE
Verifiable
Identity
Document
(SVID)

Used by a workload to
prove its identity to a
resource or caller
(JWT/X.509)



Workload
API

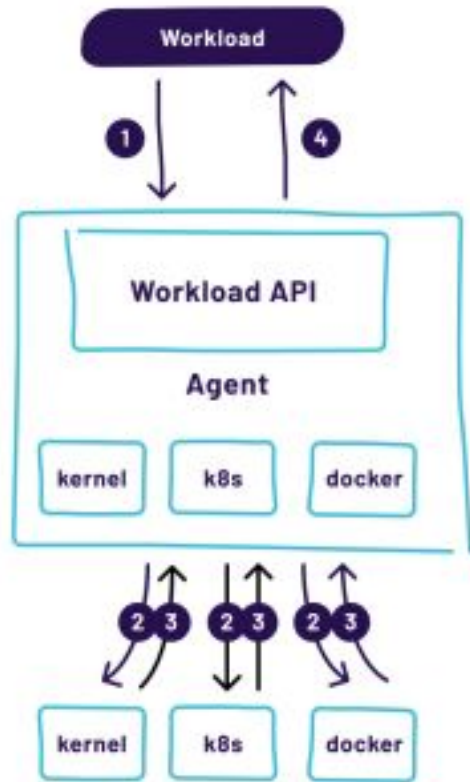
Used by a workload to
receive its identity, private
key/cert and trust bundle

Example SPIFFE ID: `spiffe://example.org/ns/bookinfo/sa/bookinfo-reviews`

Demo: SPIRE under the hood



Deep Dive: SPIRE Workload Attestation



1. A workload calls the Workload API to request an SVID.
2. The agent interrogates the node's kernel to get the attributes of the calling process.
3. The agent gets the discovered selectors (e.g. pod uid, namespace, service account,...).
4. The server determines the workload's identity by comparing discovered selectors to registration entries and returns the correct SVID to the workload.

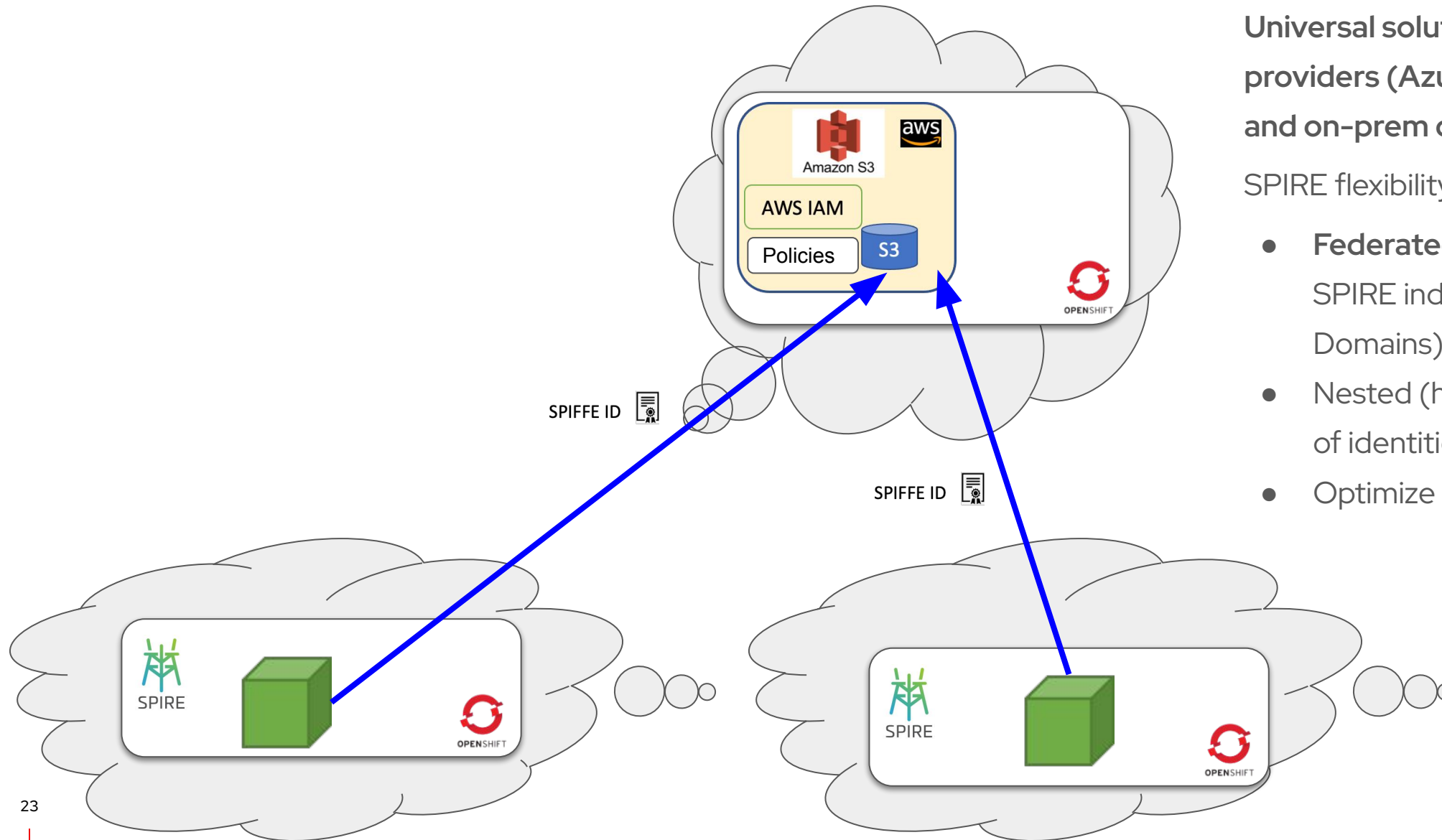
For kubernetes:

- Agent takes process id of the workload
- Agent uses local kubelet to get namespace, service account, pod uid,...
- Server matches these attributes against selectors

Example SPIRE Use Cases



Federation between Spire servers for cross cloud workload identity



Universal solution for various cloud providers (Azure, GCP, AWS, IBM) and on-prem deployments.

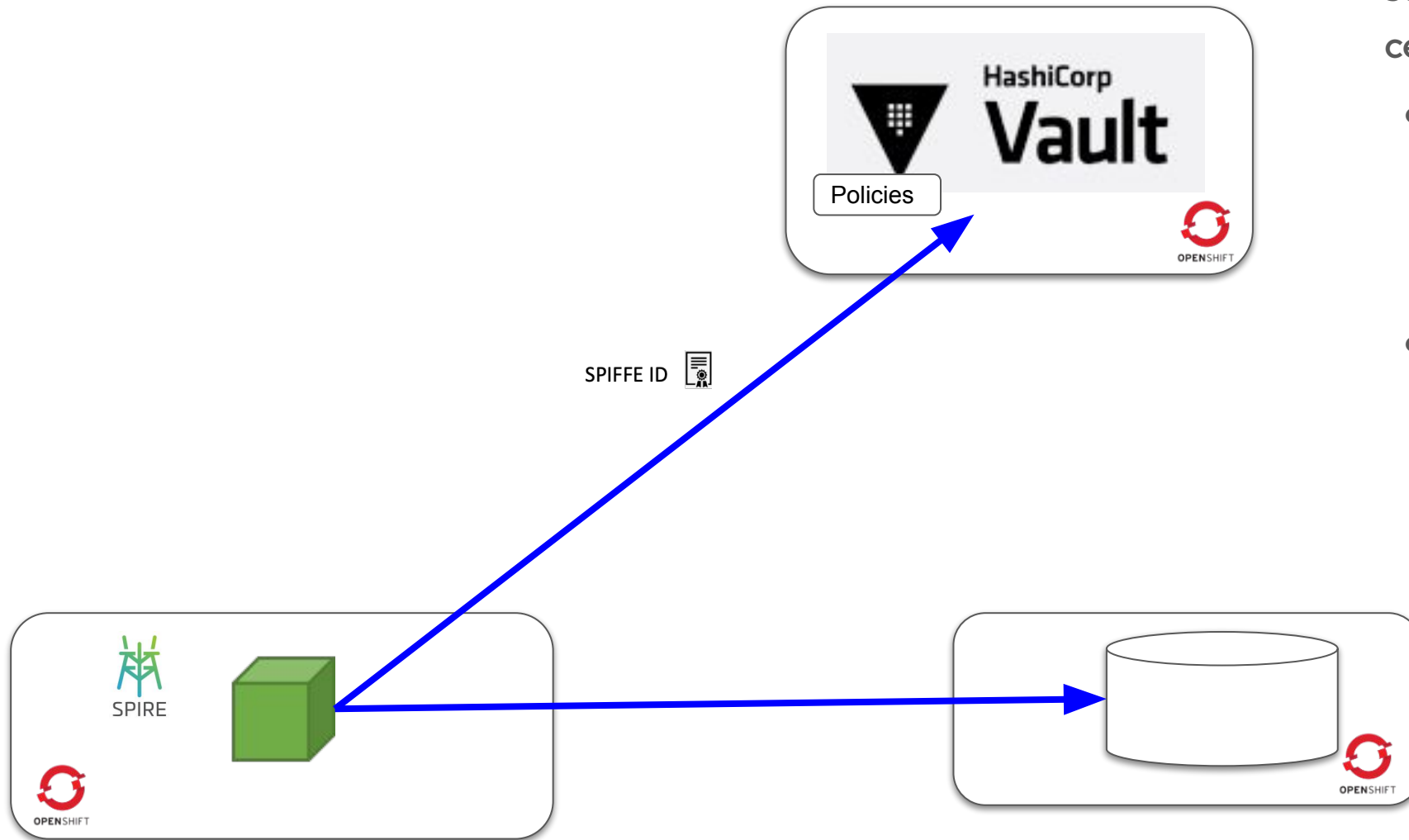
SPIRE flexibility in supporting scaling:

- **Federated Identities** (2 or more SPIRE independent Trust Domains) or
- Nested (hierarchical organization of identities)
- Optimize SPIRE deployment

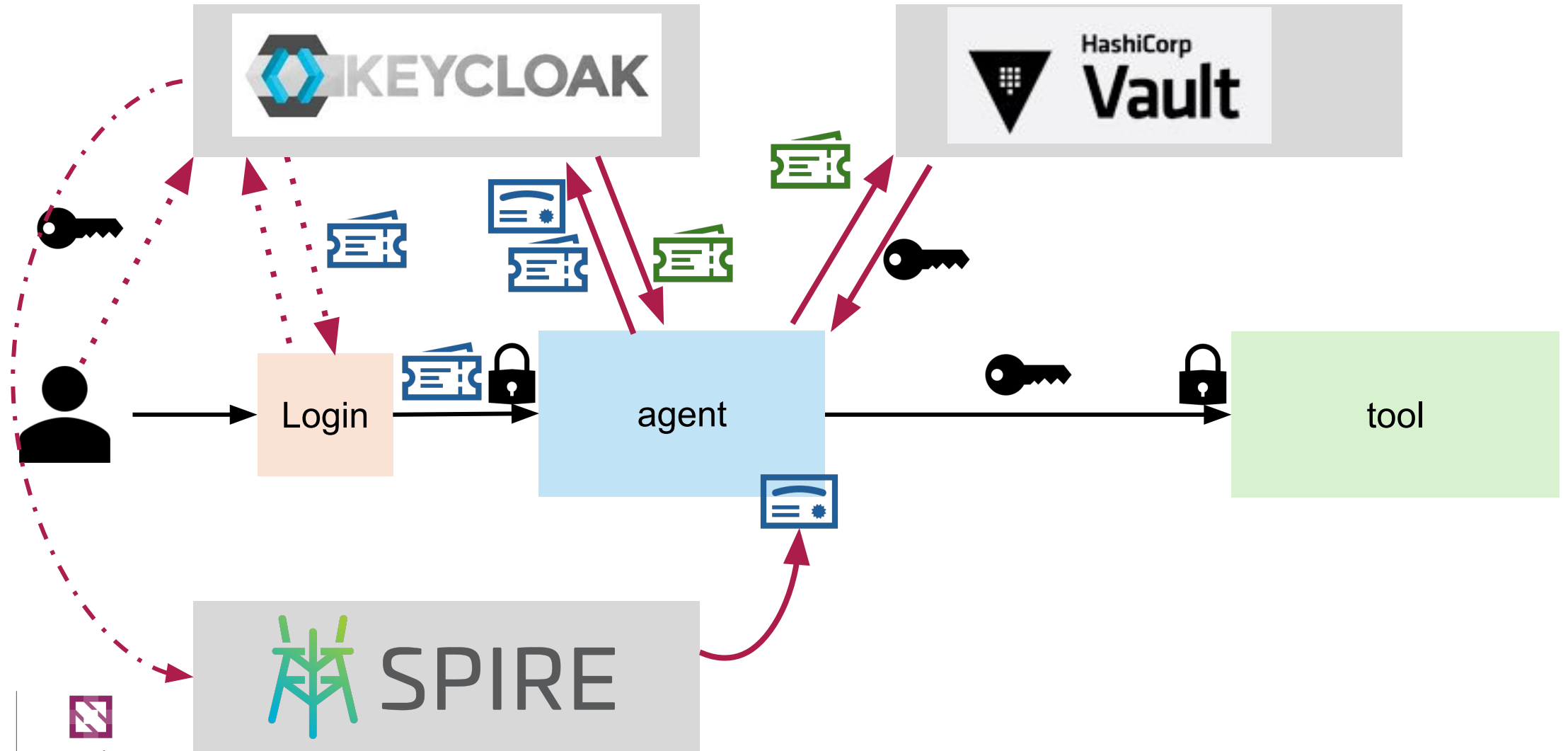
Using Spire to obtain “Secret Zero” for a Vault

“Secret Zero” is a major problem for securing infrastructures if having a central vault for storing credentials:

- SPIRE allows to retrieve the “secret zero” in order to authenticate workloads against the vault
- They can then access the required secrets based on Vault policies



Using SPIRE to secure an MCP agentic flow



Resources



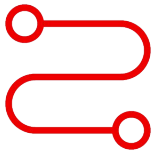
Learn more about the project!

<https://spiffe.io>



Read the blog!

<https://next.redhat.com/2024/06/27/spiffe-on-red-hat-openshift>



Explore the Learning Path!

<https://developers.redhat.com/learn/openshift/implement-cross-cloud-identity-framework-spiffespire-openshift>



Try the hands on Workshop!

<https://bit.ly/cnscon24-workload-id>



Engage in the Security SIG!

<https://commons.openshift.org/sigs>

Questions?



[linkedin.com/company/red-hat](https://www.linkedin.com/company/red-hat)



[facebook.com/redhatinc](https://www.facebook.com/redhatinc)



[youtube.com/user/RedHatVideos](https://www.youtube.com/user/RedHatVideos)



twitter.com/RedHat

Comparison Kubernetes SA vs SPIRE/SPIFFE

Feature	Kubernetes SA	SPIRE / SPIFFE
Identity	Pod name / namespace	Globally unique SPIFFE ID (<code>spiffe://<trust-domain>/<workload></code>)
Secretless authentication	Only in-cluster	Works in-cluster, multi-cluster, multi-cloud, or on-prem
Verification	RBAC in cluster	Any workload that trusts the SPIRE trust domain can verify identity
Key rotation	K8s rotates SA token	Automatic rotation of X.509 or JWT keys, fully auditable
Federation	Limited	Cross-cluster or cross-domain workload identity (federation)
Protocol-agnostic	Tied to K8s API	TLS, JWT, mTLS, SPIFFE-SVIDs, service-to-service auth

Platform	IdentityType	Static Secret	Managed Identity
 Kubernetes	Service Account	Long-lived JWT tokens	Short-lived tokens, workload identity
 Google Cloud Platform	Service Account	JSON Service Account Keys	Workload identity federation
 Amazon Web Services	IAM Role / Service Account	Access keys	IRSA, metadata service
 Azure	Managed Identity / Service Principal	Client secret/certificate	Managed identity
 GitHub Actions	OIDC Token	PAT, repository secrets	OIDC federation to cloud providers
 GitLab CI/CD	OIDC Token	Project/personal tokens	OIDC provider integration

Comparing Workload Identity w/ other approaches

Static Secrets	Cloud-provider Managed Identities	Universal Identity (e.g. SPIFFE / SPIRE)
3rd party APIs	Platform - bound	Federation
Legacy Systems	Native APIs	Attestation
Vault needed	Single Cloud	Portability

Source: <https://www.gitguardian.com/files/managed-identities-book>

Other options – primarily for Kubernetes

- Kubernetes Certificate API
- Service Mesh
- Kubernetes Service Account Tokens
- (if on the cloud) Cloud Provider Managed Identity
- OIDC Client Credentials grant type (e.g. with Keycloak)

How existing workloads can be SPIRE-enabled?

For requesting applications (generating a SVID):

- Use spire-helper as a side-car or init-container
- Use programming specific spire-libraries, e.g. [py-spiffe](#)
- Use other tools that includes this integration, e.g. cosign
- Implement it yourself (for debugging)

For receiving applications (validating a SVID):

- Use OIDC discovery end-point

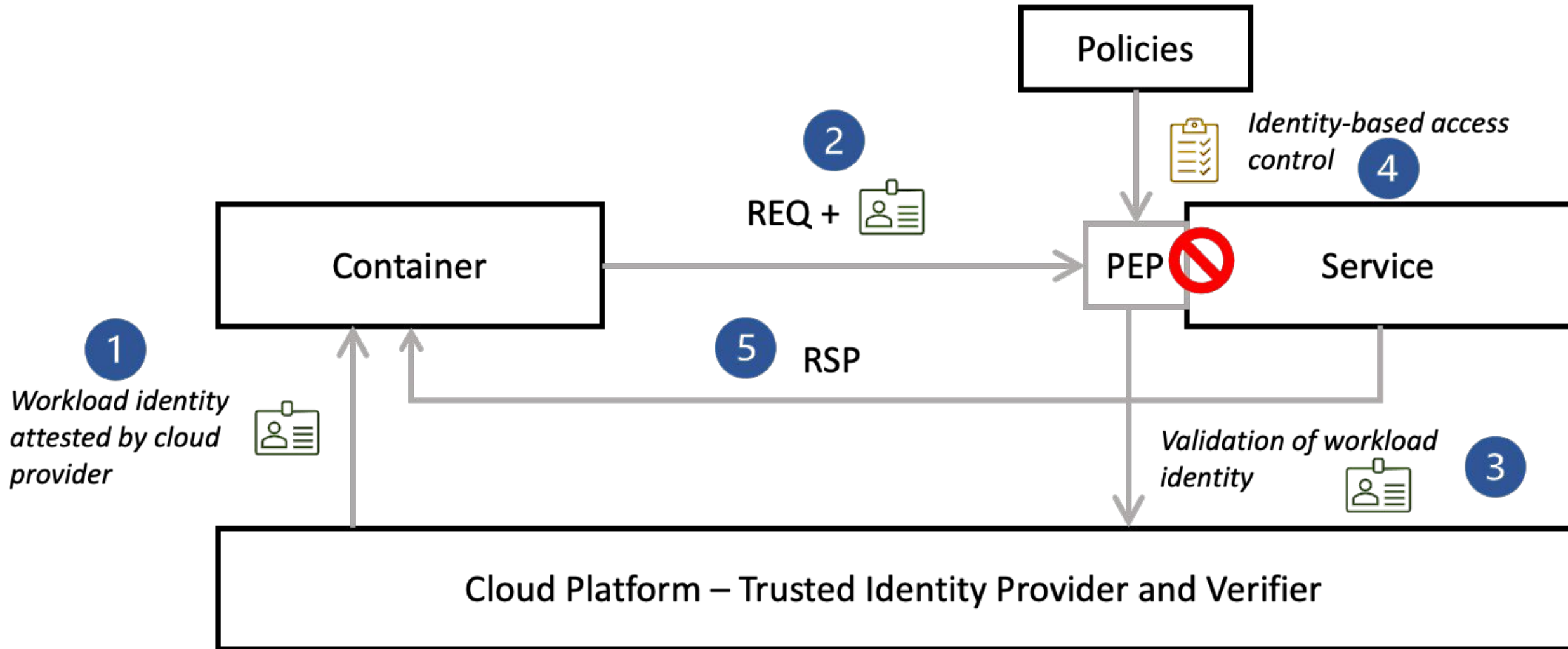
Some (philosophical) thoughts about trust

- I don't trust you, till you have proven to be a good guy (attestation)
- I don't trust what you say, but only what others say about you (kernel-node interrogation, trusted 3rd party like k8s, cloud provider,...)
- I trust what you are and not what you have (interrogation vs static keys)
- I give you a fair chance to get attested (unauthenticated unix socket)
- If I trust you once, you have to prove your good nature again and again (short-lived tokens/certificates)

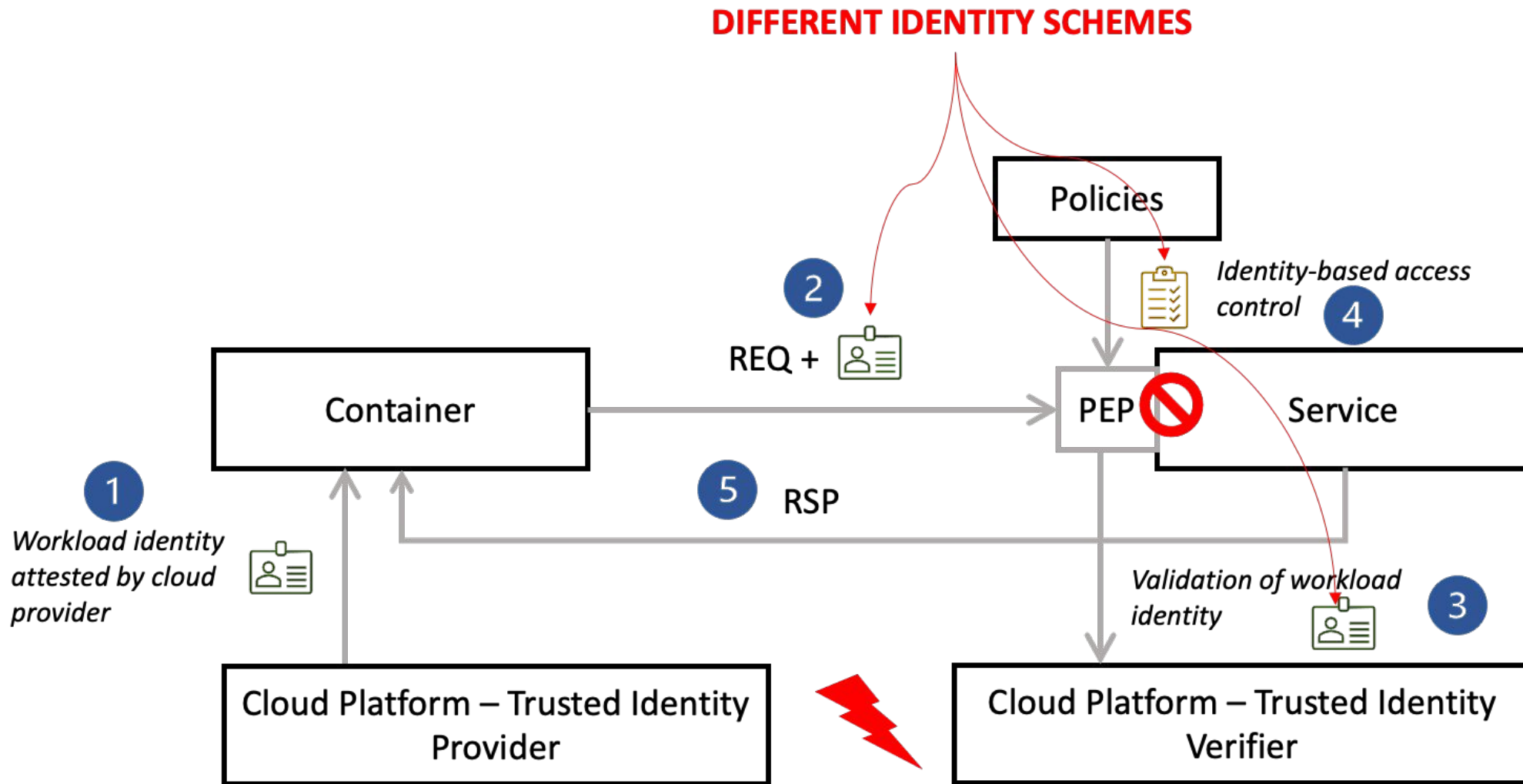
Challenges with Multi-Cloud Scenarios

- Storing **long-lived credentials** from another cloud is an easy, common solution but:
 - very difficult to account for blast radius
 - requires trusted intermediary
 - violates the Zero-Trust principles
- Cloud to cloud federation – provides **short-lived credentials**, but:
 - support for workload identity federation across clouds varies
 - different definitions across providers
 - no common schemas, difficult to establish trust relationships universal to all the clouds
 - quadratic complexity (every provider requires setup with each other)

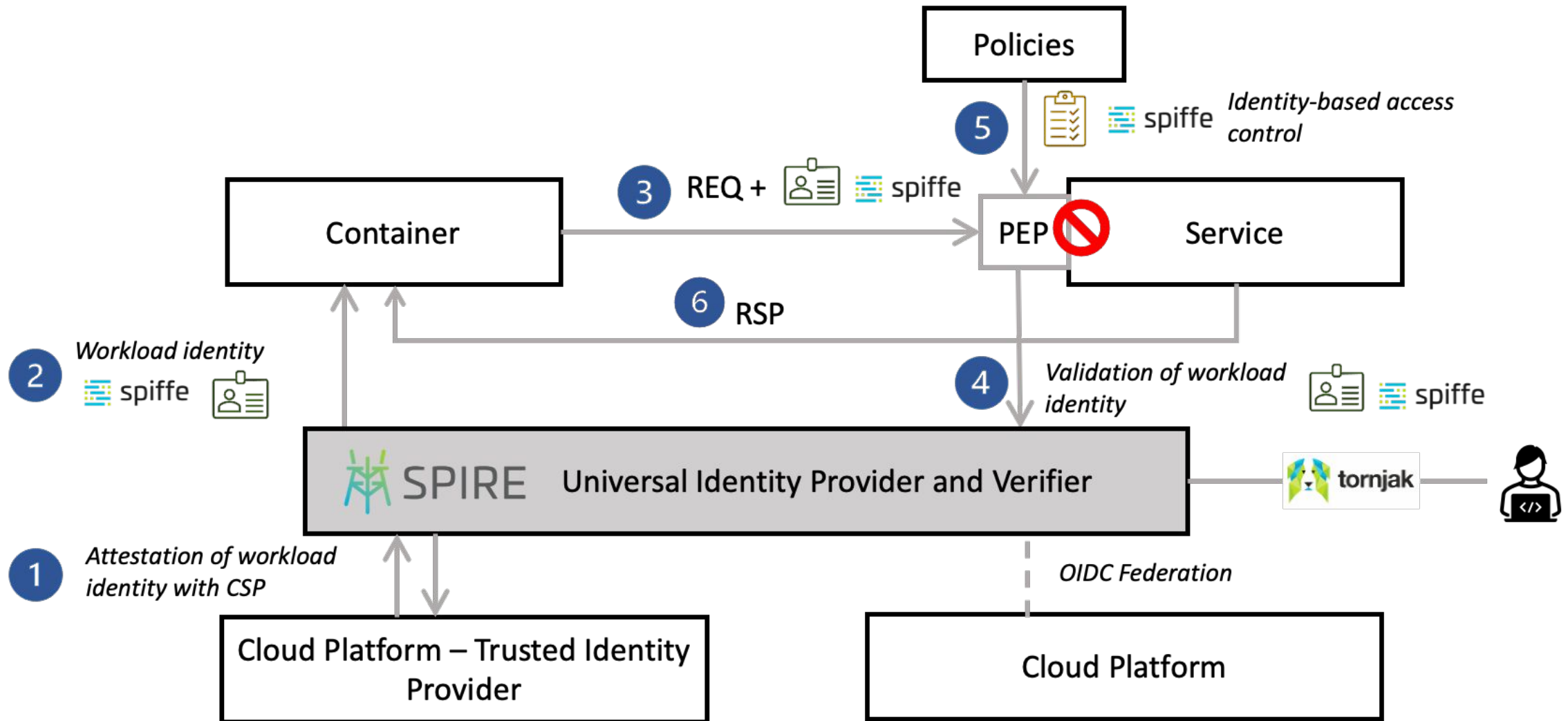
Workload Identity in single cloud is hard ...



Workload identity in multi-cloud is harder



SPIFFE, SPIRE, and Tornjak For Multi-Cloud Identity



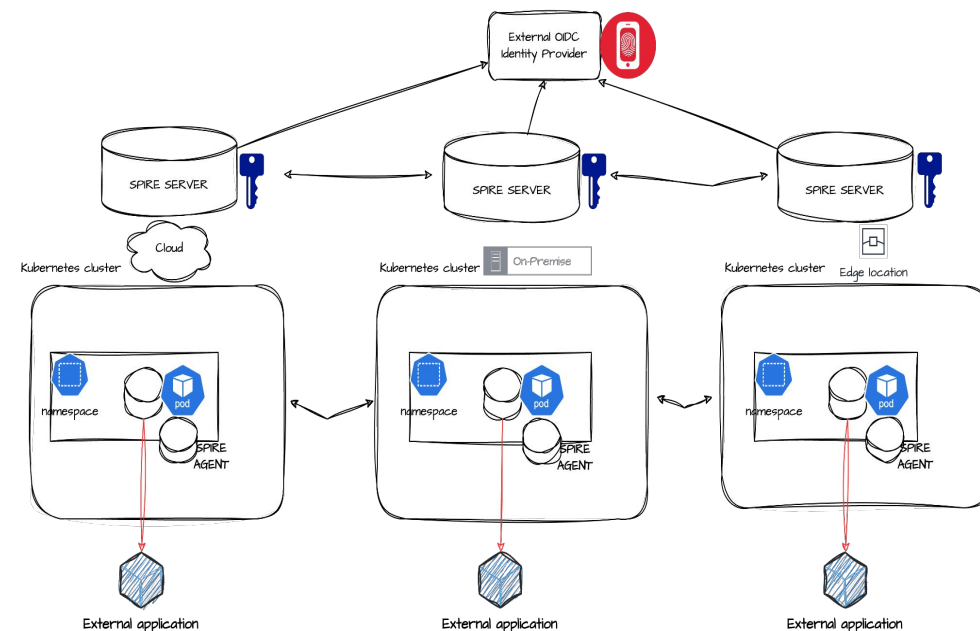
Zero Trust Workload Identity

SPIRE Initiative Tracker:

[OCPSTRAT-1691](#)

- ▶ **Enable MFA for your workloads with Red Hat's Zero Trust Workload Identity Manager Operator**
- ▶ Day 2 Operator Based on [SPIFFE/SPIRE](#)
- ▶ Single identity schema across multi-cluster deployments with federation support.
- ▶ Short-lived, cryptographically verifiable identities issued after node and workload attestation enabling zero-trust architecture
- ▶ Identities for VM and Container Workloads

Short Term (1H 2025)	Mid Term (2H 2025<>1H 2026)	Long Term (2026+)
SPIRE Operator (Tech Preview) 4.19 ➢ SPIRE Controller Manager plugin ➢ KeyManager Plugin with config options ➢ CSI driver ➢ Telemetry support ➢ Spire OIDC federation	SPIRE Operator (GA) 4.20 ➢ Integration with AWS Cloud provider ➢ Support for Tornjak for enabling UI management, Federation relationships across clouds SPIRE Operator Integration with OSSM ➢ Service Mesh with SPIRE Federation support	SPIRE Operator Integrations ➢ Multi-Cloud (Azure, GCP etc) ➢ Scaling Spire with ACM ➢ SPIRE with OCP Virt ➢ SPIRE KeyLime Integration ➢ SPIRE with TSSC ➢ SPIRE for Edge ➢ SPIRE with Kessel/Google Zanzibar ➢ SPIRE as Kubelet Image Credential provider ➢ SPIRE with CoCo ➢ SPIRE in Agentic AI



SPIRE Server certificates

	SPIRE Server Certificate	SPIRE Server TLS certificate (for Web Authn)	SPIRE Server Federation certificate (for Web Authn)	SPIRE Server Signing Certificate (CA) - aka "Trust Bundle"
Used for	SPIFFE Authentication (stronger than mTLS)	mTLS Web Authn	mTLS Web Authn	Signing SVIDs
Subject Alternative Name	spiffe://[TRUST_DOMAIN]/spire/server	DNS: pire-server.zero-trust-workload-identity-manager.svc, DNS:spire-server.zero-trust-workload-identity-manager.svc.cluster.local	DNS:federation.apps.sno.ingendeine.cloud	spiffe://[TRUST_DOMAIN]
Private Key is stored in	Doesn't exist (not entirely sure)	Secret "spire-server-serving-certs"	Secret "spire-server-federation-tls"	Keys.json
Certificate is stored	Stored in memory			Config map "spire-bundle"
Certificate can accessed externally via		openssl s_client -connect spire-server:443 -showcerts </dev/null		./spire-server bundle show
Rotated	24h (default)	24h (default)	24h (default)	24h (default)